UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS



INFORME DE GUÍA DE LABORATORIO N.º 07

PRIMALIDAD, GRUPOS Y ALGORITMOS PROBABILÍSTICOS

Asignatura: ALGORITMOS AVANZADOS

Docente: Ing. Héctor Eduardo Ugarte Rojas

Equipo: GRUPO 5 **Semestre:** 2026-II

Integrantes:

- Choquenaira Quispe, Noe Franklin 133962
- Porroa Sivana, Yeni Ruth 120893
- Quispe Rimachi, Romario 164257
- Yaranga Achahui, Aldo 103179

Cusco – Perú

2026

Índice

1. Trabajo preparatorio y fundamentos algebraicos **1**

1.1. Unidades módulo n, función φ y orden 1

1.2. División de prueba, pseudoprimos y Carmichael 1

1.3. Exponenciación binaria y Miller–Rabin 1

1.4. Probabilidad de error y tipo de algoritmo 1

1.5. Modelo de costos y expectativa de candidatos 2

2. Ejercicios resueltos **2**

2.1. Resuelto 1: división de prueba y Fermat 2

2.2. Resuelto 2: Miller–Rabin con traza verificable 3

2.3. Resuelto 3: prueba repetida y candidatos 3

3. Ejercicio propuesto 1: biblioteca de primalidad **4**

3.1. Problema e interfaz solicitada 4

3.2. Construcción paso a paso 4

3.3. Resultados reproducibles y propiedades verificadas 5

3.4. Interpretación del resultado y seguridad 6

4. Ejercicio propuesto 2: comparación experimental **6**

4.1. Diseño y metodología de medición 6

4.2. Comparación de tiempo y operaciones 6

4.3. Generación y precribado 9

4.4. Testigos, mentirosos fuertes y probabilidad de error 10

5. Respuestas a las preguntas de análisis **12**

6. Conclusiones grupales **13**

1. Trabajo preparatorio y fundamentos algebraicos

1.1. Unidades módulo n, función φ y orden

El conjunto $\mathbb{Z}_n^* = \{a \bmod n : \gcd(a,n) = 1\}$ es un grupo multiplicativo de orden $\varphi(n)$. El orden de una unidad a es el menor $t > 0$ con $a^t \equiv 1 \pmod{n}$; por el teorema de Lagrange, divide a $\varphi(n)$. Euler establece $a^{\varphi(n)} \equiv 1 \pmod{n}$ cuando $\gcd(a,n) = 1$. Si $n = p$ es primo, $\varphi(p) = p - 1$: se obtiene el pequeño teorema de Fermat [3, 2]. La contraposición permite *certificar composición* cuando falla la congruencia; el recíproco no es válido.

1.2. División de prueba, pseudoprimos y Carmichael

Si $n = ab$ es compuesto y ambos factores excedieran $\sqrt{n}$, entonces $ab > n$. Por ello basta comprobar los posibles divisores hasta $\lfloor \sqrt{n} \rfloor$. Para $L = \lceil \log_2(n+1) \rceil$ bits, esta cota es de orden $2^{L/2}$ divisiones; no es polinómica en L. Una base a coprima a un compuesto n que cumple $a^{n-1} \equiv 1 \pmod{n}$ convierte a n en un *pseudoprimo de Fermat* para a. Por ejemplo, $341 = 11 \cdot 31$ pasa la base 2. Un número de Carmichael pasa todas las bases coprimas: $561 = 3 \cdot 11 \cdot 17$, $1105 = 5 \cdot 13 \cdot 17$ y $1729 = 7 \cdot 13 \cdot 19$ son casos de prueba. El criterio de Korselt caracteriza tales números como compuestos libres de cuadrados con $(p-1) \mid (n-1)$ para todo primo $p \mid n$ [4].

1.3. Exponenciación binaria y Miller–Rabin

La representación binaria $e = \sum b_i 2^i$ permite computar a^e mód n mediante cuadrados sucesivos: $O(\log e)$ productos modulares, sin construir a^e completo. La instrumentación cuenta cuadrados *ejecutados*, incluidos los de la exponenciación inicial y la cadena posterior. Para $n > 2$ impar se descompone $n - 1 = 2^s d$ con d impar. La ronda comienza con $x = a^d$ mód n; acepta si $x = 1$ o si $x = -1 \pmod{n}$ al inicio o en alguno de los $s - 1$ cuadrados posteriores. Si la cadena alcanza 1 desde $x \not\equiv \pm 1$, se encontró una raíz no trivial de 1, imposible en un cuerpo $\mathbb{Z}_p$: se certifica composición y $\gcd(x \pm 1, n)$ puede proporcionar factores. Cuando una base rechaza al compuesto se denomina *testigo*; cuando lo acepta, *mentiroso fuerte*.

1.4. Probabilidad de error y tipo de algoritmo

Para un compuesto impar fijo, como máximo un cuarto de las bases aleatorias válidas son mentirosos fuertes. Con r bases uniformes independientes, la probabilidad de aceptar ese compuesto es a lo sumo 4^{-r}. Para $r = 1, 5, 10, 20$ las cotas son 0,25, $9{,}765625 \cdot 10^{-4}$, $9{,}536743 \cdot 10^{-7}$ y $9{,}094947 \cdot 10^{-13}$. La conclusión “compuesto” es definitiva; “probable primo” no es una prueba matemática. Miller–Rabin es Monte Carlo de error unilateral: tiempo acotado por rondas y posible falso positivo, nunca falso negativo para un primo. Un algoritmo Las

Vegas siempre entrega una respuesta correcta, pero su tiempo de ejecución puede depender del azar. Las semillas fijas reproducen los experimentos; para usos criptográficos se requiere `secrets.SystemRandom` y entropía no predecible [5].

1.5. Modelo de costos y expectativa de candidatos

Por ronda, Fermat y Miller–Rabin requieren $O(L)$ productos modulares; con r rondas, Miller–Rabin emplea $O(rL)$ productos, frente a $O(2^{L/2})$ divisiones de prueba. Para multiplicación escolar de enteros de L bits, la estimación de costo por ronda es $O(L^3)$ operaciones de bit. Entre impares de L bits, la densidad aproximada de primos es $2/(L\ln 2)$: se esperan alrededor de $L\ln 2/2$ candidatos impares por primo. El precribado evita exponenciaciones innecesarias, pero no cambia la probabilidad de que un candidato generado al azar sea primo.

2. Ejercicios resueltos

El archivo `python/ejercicios_resueltos.py` contiene los tres ejercicios ejecutables y comprueba los valores con aserciones.

2.1. Resuelto 1: división de prueba y Fermat

La división de prueba trata $n < 2$, 2, 3, pares y divisores impares hasta `isqrt(n)`. Fermat verifica primero $\gcd(a,n)$ y después computa a^{n-1} mód n. El siguiente código utiliza las funciones de `primality.py` y exhibe los tres casos pedidos:

```python
def resuelto_1():
    """División de prueba y contraejemplo al recíproco de Fermat."""
    division_221 = trial_division(221)
    fermat_15 = fermat_round(15, 2)
    fermat_341 = fermat_round(341, 2)
    assert division_221 is False and fermat_15 is False and fermat_341 is True
    return {"221_es_primo": division_221, "15_pasa_fermat": fermat_15,
            "341_pasa_fermat": fermat_341}
```

Listing 1: División de prueba y Fermat: código ejecutable

Resultado de la ejecución:

```
>>> resuelto_1()
{'221_es_primo': False, '15_pasa_fermat': False, '341_pasa_fermat': True}
```

Aquí $341 = 11 \cdot 31$ pasa Fermat base 2, aunque es compuesto. Para 561, la base 2 también pasa Fermat por ser Carmichael; la base 3 comparte factor con 561 y certifica composición.

2.2. Resuelto 2: Miller–Rabin con traza verificable

El par (s,d) de $221-1=220=2^2\cdot 55$ y la cadena de residuos de la base 2 son:

$$2^{55} \bmod 221 = 128, \qquad 128^2 \bmod 221 = 30 \neq 220. \tag{1}$$

No aparece $-1 \equiv 220$: la base 2 demuestra que 221 es compuesto. La salida incluye n, a, s, d, residuos, motivo y contadores de operaciones:

```python
def resuelto_2():
    """Traza verificable de Miller--Rabin para n=221, base a=2."""
    assert decompose(221) == (2, 55)
    passed, detail = miller_rabin_round(221, 2, trace=True)
    assert not passed and detail["residuos"] == [128, 30]
    assert (detail["n"], detail["a"], detail["s"], detail["d"]) == (221, 2, 2, 55)
    return {"pasa": passed, **detail}
```

Listing 2: Miller–Rabin con traza: código ejecutable

Resultado de la ejecución:

```
>>> resuelto_2()
{'pasa': False,
 'n': 221,
 'a': 2,
 's': 2,
 'd': 55,
 'residuos': [128, 30],
 'cuadrados_cadena': 1,
 'cuadrados_modulares': 6,
 'productos_modulares': 5,
 'motivo': 'sin_menos_uno'}
```

Se obtuvieron además las trazas exigidas: para $(n,a)=(341,2)$, $s=2$, $d=85$, cadena $[32,1]$; para $(561,2)$, $s=4$, $d=35$, cadena $[263,166,67,1]$. Ambas se rechazan por una raíz no trivial de 1. En 341, $\gcd(32-1,341)=31$ y $\gcd(32+1,341)=11$. Todas las trazas, también las de 1105, 1729 y un mentiroso fuerte de 25, están en `results/trazas_miller_rabin.csv`.

2.3. Resuelto 3: prueba repetida y candidatos

```python
def resuelto_3():
    """Candidato reproducible y cinco rondas sobre el primo conocido 1009."""
    rng = Random(2026)
    n = generate_candidate(32, rng)
    assert n.bit_length() == 32 and n % 2 == 1
    report = primality_report(1009, rounds=5, rng=Random(2026))
    assert report["resultado"] == "probable_primo"
    assert report["rondas_ejecutadas"] == 5 and report["cota_error"] == 4.0**-5
    return {"candidato": n, "bits": n.bit_length(), "impar": bool(n % 2),
            "1009_resultado": report["resultado"],
            "rondas_ejecutadas": report["rondas_ejecutadas"],
            "cota_error": report["cota_error"]}
```

Listing 3: Prueba repetida y candidatos: código ejecutable

Resultado de la ejecución:

```
>>> resuelto_3()
{'candidato': 2659099673,
 'bits': 32,
 'impar': True,
 '1009_resultado': 'probable_primo',
 'rondas_ejecutadas': 5,
```

```
 'cota_error': 0.0009765625}
```

Los bits superior e inferior se fijan a 1, respectivamente para garantizar longitud exacta e imparidad. Se eligió el primo conocido 1009, mayor que los primos del precribado, para que se ejecuten realmente las cinco rondas: su resultado es "probable primo" y la cota condicional es $4^{-5} = 1/1024$. Con semilla fija la ejecución es reproducible, pero *no* segura para generar claves.

3. Ejercicio propuesto 1: biblioteca de primalidad

3.1. Problema e interfaz solicitada

Se integraron en `python/primality.py` las siete funciones de la guía. El objetivo no es confundir "pasa una ronda" con "es primo": una base testigo prueba composición, mientras que un candidato que supera las rondas se informa como probable primo. La interfaz pública y el tipo de resultado de cada operación se presentan en el Cuadro 1.

Tabla 1: Funciones de la biblioteca y sus contratos

Función	Entrada, salida y convención
`trial_division(n)`	Booleano determinista; $n < 2$ devuelve `False`.
`fermat_round(n,a)`	Booleano con $2 \le a \le n-2$; `False` certifica compuesto.
`decompose(n)`	Para $n \ge 3$ impar, devuelve (s,d) con $n-1 = 2^s d$ y d impar.
`miller_rabin_round(n,a,trace)`	Devuelve $(pasa, detalle)$; el detalle registra n,a,s,d, residuos opcionales y contadores.
`is_probable_prime(n,rounds,rng)`	Respuesta booleana con precribado y bases aleatorias.
`generate_candidate(bits,rng)`	Entero impar de exactamente $bits \ge 2$ bits.
`generate_probable_prime(bits,rounds)`	Muestrea candidatos hasta encontrar uno aceptado; puede devolver contadores.

3.2. Construcción paso a paso

1. **Entradas y casos límite.** Se rechazan tipos no enteros, bases fuera de $[2, n-2]$ y descomposiciones no válidas; 0 y 1 no son primos ni compuestos. La división de prueba llega hasta `isqrt(n)`.

2. **Fermat configurable.** `fermat_test` acepta una secuencia de bases; primero comprueba $\gcd(a,n)$ y luego la congruencia. 341 supera la base 2 pese a ser compuesto.

3. **Miller–Rabin auditable.** Se escribe $n-1=2^s d$, se calcula a^d mód n y se registra cada residuo de la cadena hasta hallar -1, una raíz no trivial de 1 o un testigo. La exponenciación binaria propia cuenta cuadrados realizados; no se infiere el trabajo interno de `pow()`.

4. **Precribado y generación.** Una criba construye los primos hasta 997. Si el candidato coincide con uno de ellos, queda certificado; si es divisible por alguno, se descarta. Los bits superior e inferior del candidato se fijan a uno. Se prueban nuevos candidatos hasta superar las rondas solicitadas.

3.3. Resultados reproducibles y propiedades verificadas

El Cuadro 2 sale de `casos_conocidos.csv`. En las columnas de Fermat y Miller–Rabin, "sí" significa *supera la base 2*, no primalidad certificada; para $n < 5$ se señala que la ronda no aplica. En 341, 561, 1105 y 1729 Fermat pasa, pero Miller–Rabin con base 2 encuentra un testigo.

Tabla 2: Casos especiales y pseudoprimos: resultado de las tres pruebas

n	Clasificación	División	Fermat (2)	MR (2)
0	ni primo ni compuesto	no	–	–
1	ni primo ni compuesto	no	–	–
2	primo	sí	–	–
3	primo	sí	–	–
25	compuesto	no	no	no
341	compuesto	no	sí	no
561	compuesto	no	sí	no
1105	compuesto	no	sí	no
1729	compuesto	no	sí	no

La ronda devuelve también n, a, s, d y residuos. Los tres casos exigidos se cotejaron con `trazas_miller_rabin.csv` y se presentan en el Cuadro 3. Las cadenas de 341 y 561 terminan en una raíz no trivial de 1, lo que demuestra composición; para 221 no aparece -1.

Tabla 3: Trazas verificables de Miller–Rabin con base 2

n	a	s	d	Residuos	Decisión
221	2	2	55	[128, 30]	testigo
341	2	2	85	[32, 1]	testigo
561	2	4	35	[263, 166, 67, 1]	testigo

La suite `python/test_primality.py` ejecuta 32 pruebas: los primeros 10 001 enteros frente a una criba independiente; 1000 valores aleatorios menores que 10^6 frente a división de prueba; cuadrados de primos, extremos de bases y distintas rondas. `classify_bases` clasifica exhaustivamente cada base válida de los compuestos impares $5 \leq n < 200$ y de 221, 341 y 561. Se comprobó que todo primo pequeño supera todas sus bases válidas y que los candidatos de 16 a 1024 bits son impares y conservan la longitud solicitada.

3.4. Interpretación del resultado y seguridad

`primality_report` registra resultado, rondas solicitadas y ejecutadas, bases, cuadrados y descarte por precribado. Un compuesto detectado tiene error cero; para un compuesto impar que *podría* superar r bases elegidas uniformemente e independientemente, la cota es 4^{-r}. Una lista de bases fija o una semilla reproducible no constituye una garantía criptográfica para esa ejecución. En ausencia de `rng`, el muestreo usa `secrets.SystemRandom`; `random.Random(seed)` se reserva para comparar experimentos y nunca para claves.

4. Ejercicio propuesto 2: comparación experimental

4.1. Diseño y metodología de medición

Se estudiaron longitudes de 16, 32, 64, 128, 256, 512 y 1024 bits. Las semillas de generación, comparación y simulación fueron `20260701`, `20260702` y `20260703`. Por tamaño se generaron 30 probables primos y, para comparar pruebas, se prepararon 8 candidatos aceptados y 8 compuestos impares construidos como producto de dos factores mayores que uno. Los candidatos de 16 y 32 bits se verificaron además por división de prueba; los mayores no se consideran primos certificados.

Las entradas y 20 bases válidas por entrada se seleccionaron antes de medir. Fermat y Miller–Rabin evaluaron las mismas entradas y bases iniciales con 1, 5, 10 y 20 rondas; la división de prueba se limitó a 16 y 32 bits. Cada medición se repitió siete veces con `perf_counter_ns()`, sin impresión, selección de bases ni escritura de archivos dentro del intervalo medido. Se registraron mínimo, mediana, máximo, rondas y cuadrados modulares. El precribado se desactivó *solo* en la comparación para observar el costo de las pruebas; durante la generación permaneció activo y se contaron sus descartes.

4.2. Comparación de tiempo y operaciones

El Cuadro 4 compara las medianas por tipo de entrada. El Cuadro 5 presenta mínimo, mediana y máximo de siete repeticiones. Los registros individuales y las bases utilizadas están

en `python/results/comparacion_metodos.csv` y `python/results/entradas_y_bases.csv`.

Tabla 4: Tiempo mediano por prueba sobre probables primos (µs)

Bits	División	Fermat 20	MR 1	MR 5	MR 20
16	17.1	173.4	14.2	56.7	209.3
32	6177.6	564.2	33.1	149.2	606.2
64	–	1366.2	70.8	372.6	1319.2
128	–	3560.7	149.7	927.5	2947.2
256	–	10164.4	434.1	2639.2	10243.0
512	–	44926.6	2300.7	10997.1	44781.4
1024	–	221081.4	10763.3	53382.4	228012.8

Tabla 5: Mínimo, mediana y máximo de siete pruebas sobre una misma entrada (μs)

Bits	Método	Mínimo	Mediana	Máximo
16	División	17.6	17.9	19.0
16	MR 20	207.3	217.4	241.8
32	División	3746.7	4249.9	13950.4
32	MR 20	406.6	650.2	672.7
1024	MR 20	209706.7	224301.2	226631.5

Lectura cuantitativa del costo y error. Para 1024 bits, la mediana de Miller–Rabin con 20 rondas fue 228012.8 μs frente a 10763.3 μs con una ronda (razón 21.2); Fermat con 20 rondas tomó 221081.4 μs. El compuesto 341 presentó 48 mentirosos fuertes entre 338 bases válidas: $p_{341} = 0{,}1420$. El tiempo medido de Fermat no resuelve sus falsos positivos para Carmichael; cero errores observados tras varias rondas no implica probabilidad cero.

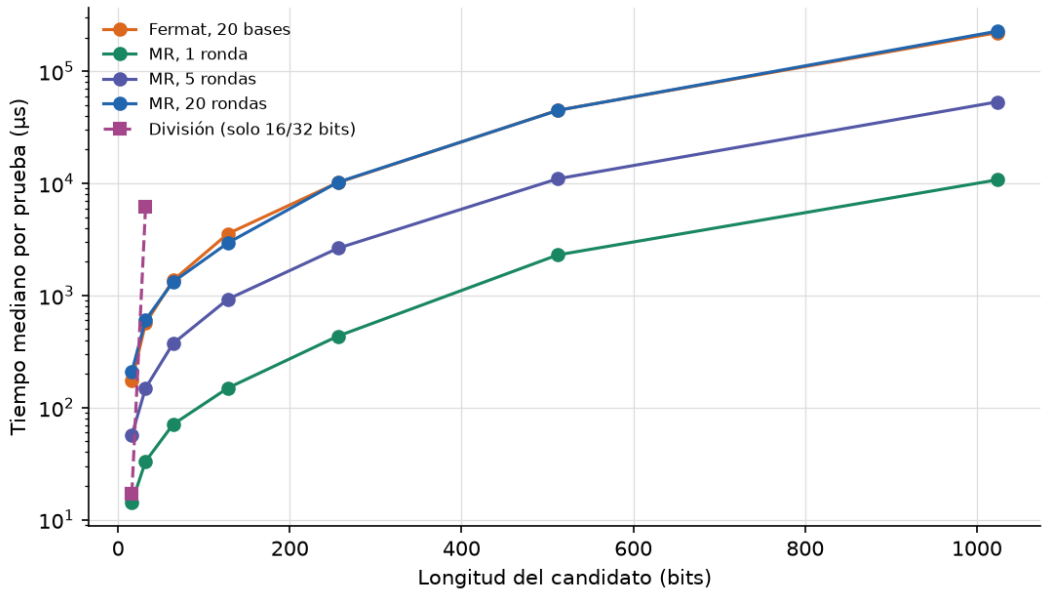


Figura 1: Tiempo mediano por prueba y longitud binaria; escala temporal logarítmica. La división se ejecuta solo hasta 32 bits.

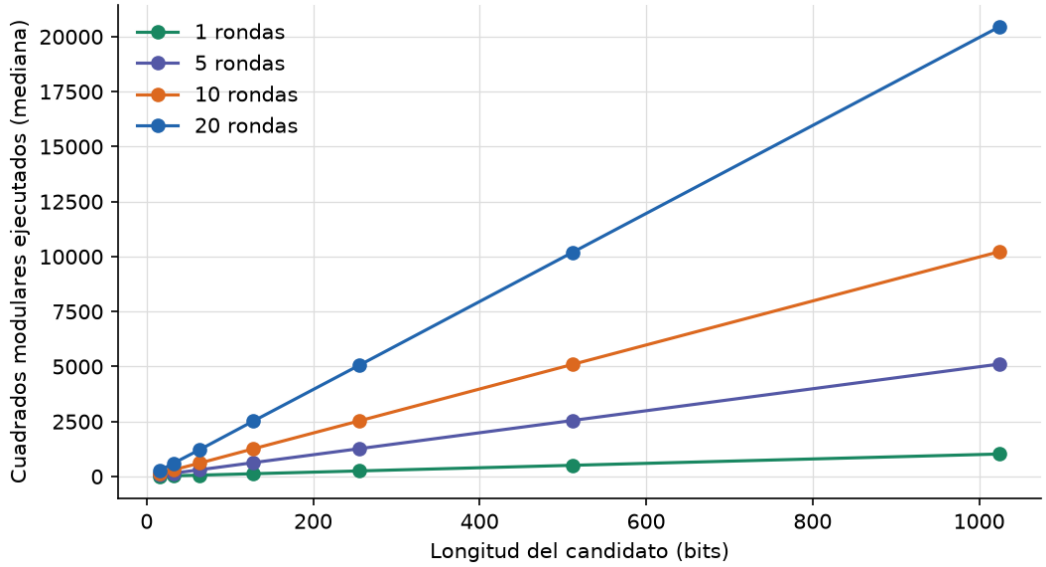


Figura 2: Cuadrados modulares efectivamente realizados por Miller–Rabin con 1, 5, 10 y 20 rondas.

Las rondas terminan temprano si se encuentra un testigo de composición. Sobre candidatos aceptados se completan todas las rondas; por ello es necesario distinguir el costo de una prueba individual del costo total de buscar un probable primo.

4.3. Generación y precribado

El Cuadro 6 resume 30 generaciones por tamaño: candidatos, descartes y tiempo. La semilla y los contadores individuales constan en `python/results/generacion_210_probables_primos.csv`.

Tabla 6: Generación de 30 probables primos por longitud binaria

Bits	Candidatos	Media	Mediana	Descartes	Tiempo med. (ms)
16	169	5.63	5.0	139	0.220
32	371	12.37	10.5	310	0.720
64	652	21.73	16.5	537	1.367
128	970	32.33	22.5	806	3.181
256	1840	61.33	40.5	1541	10.599
512	5307	176.90	97.5	4445	69.182
1024	12870	429.00	314.5	10833	802.315

Lectura cuantitativa de la generación. Para 1024 bits se necesitaron 12870 candidatos para 30 probables primos (media 429.00, mediana 314.5); el precribado descartó 10833 (84.2 %). La referencia asintótica es 354.9 candidatos impares por primo. Para 512 bits, la media observada fue 176.90 frente a 177.4 estimados. Las diferencias con la aproximación pueden reflejar variación muestral; por sí solas no prueban ni descartan errores de primalidad.

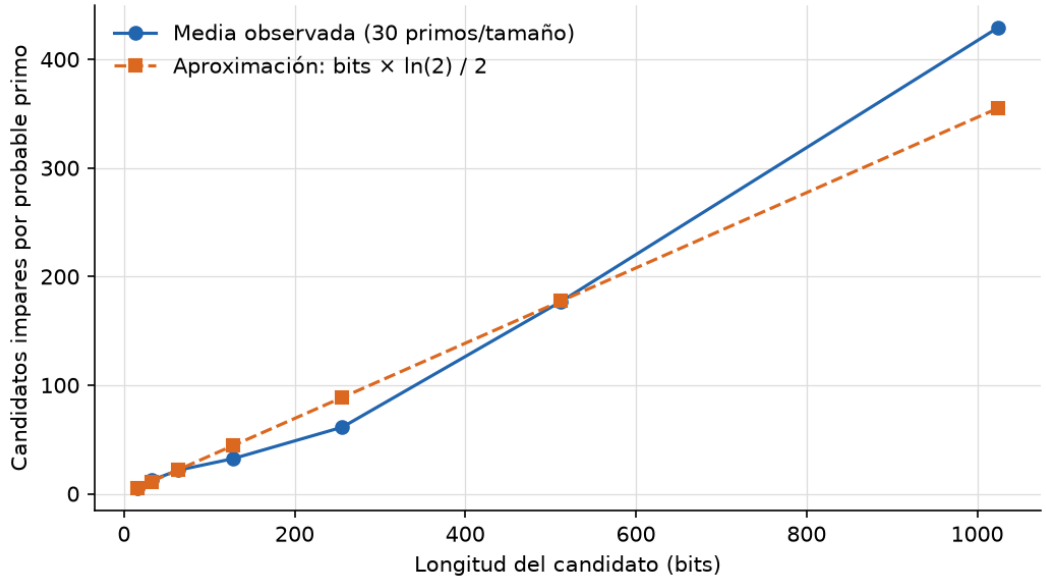


Figura 3: Candidatos impares por probable primo: media de 30 muestras y aproximación $L\ln(2)/2$.

El precribado evita exponenciaciones en compuestos con factores pequeños, pero no cambia la densidad de primos entre todos los candidatos originalmente muestreados.

4.4. Testigos, mentirosos fuertes y probabilidad de error

El Cuadro 7 enumera la cantidad de bases testigo y mentirosas fuertes; el archivo `clasificacion_bases.csv` identifica cada base válida del rango pequeño. Para un compuesto impar fijo, si p_n es la fracción de mentirosos fuertes, la probabilidad exacta de pasar r bases elegidas uniformemente con reemplazo es p_n^r. La cota universal 4^{-r} es un límite superior, no la frecuencia esperada en una muestra concreta.

Tabla 7: Testigos y mentirosos fuertes: bases válidas de compuestos seleccionados

n	Bases	Testigos	Mentirosos	Fracción
25	22	20	2	0.0909
91	88	72	16	0.1818
121	118	110	8	0.0678
221	218	214	4	0.0183
341	338	290	48	0.1420
561	558	550	8	0.0143

Tabla 8: Aceptación empírica y exacta de compuestos para 3000 ensayos por fila

n	Rondas	Aceptados	Frecuencia	Exacta p_n^r	Cota 4^{-r}
25	1	281	0.0937	0.0909	0.25
25	5	0	0	6.21e-06	0.000977
25	20	0	0	1.49e-21	9.09e-13
341	1	413	0.138	0.142	0.25
341	5	0	0	5.78e-05	0.000977
341	20	0	0	1.11e-17	9.09e-13
561	1	38	0.0127	0.0143	0.25
561	5	0	0	6.06e-10	0.000977
561	20	0	0	1.35e-37	9.09e-13

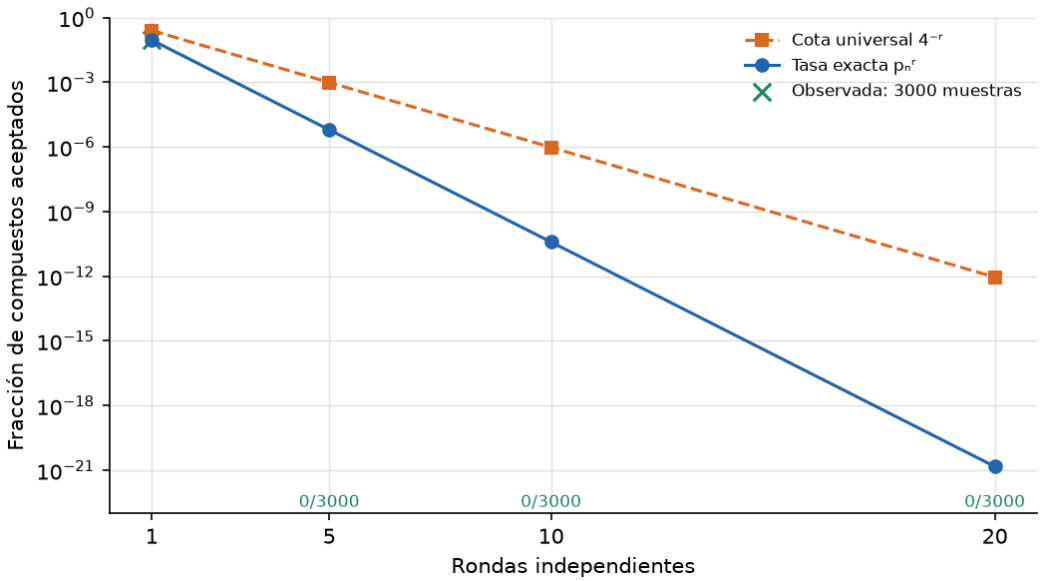


Figura 4: Compuesto 25: tasa exacta p_n^r, cota 4^{-r} y 3000 ensayos por número de rondas. Los ceros observados se anotan como $0/3000$, no como probabilidades positivas.

La simulación consulta la clasificación exhaustiva de bases de $n \in \{25, 91, 121, 341, 561\}$ para $r \in \{1, 5, 10, 20\}$; no se interpreta como una medición del tiempo de Miller–Rabin. No observar falsos positivos en una muestra finita no demuestra error nulo. Fermat no dispone de una cota análoga para números de Carmichael.

5. Respuestas a las preguntas de análisis

1. **¿Por qué basta probar divisores hasta $\sqrt{n}$?** Si $n = ab$ y $a > \sqrt{n}$, $b > \sqrt{n}$, entonces $ab > n$, contradicción. Todo compuesto tiene al menos un divisor $\leq \sqrt{n}$.

2. **¿Por qué esa cota es exponencial en bits?** Para L bits, n es del orden de 2^L y $\sqrt{n}$ del orden de $2^{L/2}$. Una función exponencial de L no es polinómica aunque sea sublineal en n.

3. **¿Por qué Fermat puede demostrar composición, pero no primalidad?** Para un primo p y base coprima a, Fermat exige $a^{p-1} \equiv 1 \pmod{p}$. Si falla, por contraposición n es compuesto; si pasa, puede ser un pseudoprimo. El ejemplo 341 con base 2 confirma que el recíproco es falso.

4. **¿Qué diferencia hay entre pseudoprimo y número de Carmichael?** Un pseudoprimo compuesto pasa Fermat para una base específica coprima; Carmichael pasa para *todas* las bases coprimas. 341 pasa base 2, mientras 561 pasa cualquier base coprima. La base 3 no es coprima a 561 y sí lo detecta.

5. **¿Por qué Miller–Rabin examina $n-1 = 2^s d$?** Separar la máxima potencia de 2 produce la cadena $a^d, a^{2d}, \ldots, a^{2^{s-1}d}$. En un módulo primo, si el último término es 1, la cadena debe comenzar en 1 o alcanzar -1 antes: solo ± 1 son raíces de $x^2 = 1$ en un cuerpo.

6. **¿Qué significa encontrar una raíz no trivial de 1?** Existe $x \not\equiv \pm 1 \pmod{n}$ con $x^2 \equiv 1 \pmod{n}$. Como $(x-1)(x+1) \equiv 0 \pmod{n}$ sin que uno de esos factores sea $0 \pmod{n}$, n es compuesto. En 341, $x = 32$ y los MCD respectivos recuperan 31 y 11.

7. **¿Qué diferencia hay entre testigo y mentiroso fuerte?** Testigo: base cuyo resultado Miller–Rabin es negativo, prueba que el número es compuesto. Mentiroso fuerte: base cuyo resultado es positivo pese a que n es compuesto. El par $(341, 2)$ es testigo, aunque 2 sea mentiroso para Fermat en 341.

8. **¿Por qué el error es unilateral?** Toda base válida hace pasar a un número primo. El rechazo significa compuesto con certeza; la única posible equivocación es aceptar como probable primo un compuesto que superó las bases elegidas.

9. **¿Qué hipótesis permite usar 4^{-r}?** El número evaluado debe ser compuesto impar; cada base se selecciona uniforme e independientemente entre las bases válidas. La fracción de mentirosos fuertes es $\leq 1/4$ y el evento de r aceptaciones tiene probabilidad $\leq (1/4)^r$. La semilla fija permite reproducir un experimento, no garantiza aleatoriedad criptográfica.

10. **¿Por qué una semilla fija no es apropiada para generar claves?** Permite repetir la misma secuencia de candidatos y bases. Si se conoce o adivina, un adversario reconstruye los valores generados. Se debe usar una fuente segura e impredecible como `secrets.SystemRandom`.

11. **¿Cómo influye el precribado?** Detecta compuestos divisibles por primos pequeños con divisiones baratas, evitando muchas exponenciaciones. También debe aceptar cuando el candidato coincide exactamente con uno de esos primos. Acelera la generación, pero no certifica primalidad de números grandes que lo superan.

12. **¿Por qué "no observamos errores" no significa "el algoritmo no puede errar"?** Una muestra finita puede omitir las bases mentirosas. La ausencia observada no prueba probabilidad cero; hay que distinguir frecuencia empírica, probabilidad exacta p_n^r para un compuesto dado y cota universal 4^{-r}.

6. Conclusiones grupales

La implementación de la Guía 07 permitió relacionar los fundamentos algebraicos con resultados verificables. La división de prueba ofrece certeza, pero su límite de aproximadamente $2^{L/2}$ divisiones para entradas de L bits impide aplicarla a tamaños criptográficos. Fermat reduce el costo mediante exponenciación modular, aunque su aceptación no demuestra primalidad: 341 supera la base 2, y los números de Carmichael 561, 1105 y 1729 superan todas las bases coprimas. Por ello, el equipo separó explícitamente los resultados "compuesto", "primo certificado" y "probable primo".

Miller–Rabin mostró una ventaja esencial: la cadena obtenida de $n-1 = 2^s d$ puede hallar testigos que Fermat no encuentra. En particular, la base 2 rechaza a 341 al descubrir la raíz no trivial 32 de 1 módulo 341; los máximos comunes divisores con 31 y 33 recuperan sus factores. Las trazas registran parámetros, residuos y operaciones reales, haciendo auditables las decisiones. Comprobamos las propiedades con una suite de 32 pruebas, una criba independiente para enteros pequeños y 1000 enteros aleatorios menores que un millón contrastados con división de prueba.

Los experimentos recorren siete longitudes entre 16 y 1024 bits y producen 210 probables primos, con candidatos, descartes por precribado, rondas, cuadrados y tiempos registrados. La comparación usa las mismas entradas y bases preseleccionadas para Fermat y Miller–Rabin, evitando atribuir al azar diferencias de rendimiento. Los candidatos de 16 y 32 bits también se certificaron con división de prueba; los mayores mantienen la condición de probables primos. Medir tiempo y contar operaciones son tareas distintas: Python maneja enteros de longitud creciente y su costo no depende solo del número de rondas.

Finalmente, una tasa empírica de error nula no implica imposibilidad de error. La cota 4^{-r} exige bases independientes uniformes, y una semilla fija es útil para reproducibilidad, pero inapropiada para claves. Un precribado correcto ahorra exponenciaciones sin transformar por sí solo una prueba probabilística en determinista. El producto entregado conserva código, pruebas, CSV, gráficos e informe reproducibles para permitir repetir y discutir cada conclusión.

Referencias

[1] H. E. Ugarte Rojas, *Guía de Laboratorio N.º 07: Primalidad, grupos y algoritmos probabilísticos*, Algoritmos Avanzados, UNSAAC, 2026-II.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022.

[3] V. Shoup, *A Computational Introduction to Number Theory and Algebra*, 2.ª ed. Cambridge University Press, 2009.

[4] R. Crandall y C. Pomerance, *Prime Numbers: A Computational Perspective*, 2.ª ed. Springer, 2005.

[5] Python Software Foundation, *secrets — Generate secure random numbers*. https://docs.python.org/3/library/secrets.html.